



VELORA MOBITECH

Kriti - Techboard IITG

Final Submission Report

Hostel Id 2633

Contents

1	Problem Statement Overview	3
1.1	Operational Constraints (Hard Constraints)	3
1.2	Soft Constraints and Preferences	3
1.3	Objective Structure	4
1.3.1	Stage 1: Employee Drop-Off Maximization	4
1.3.2	Stage 2: Cost and Time Optimization	4
1.4	Assumptions	4
2	MODELLING: Algorithmic Approaches and Implementation	5
2.1	Introduction to Our Optimization Framework	5
2.2	The Execution Dispatcher (<code>solver.py</code>)	5
2.2.1	The Final Tie-Breaker (<code>get_feasibility_score</code>)	5
2.3	The Core Engine: <code>LNSOptimizer</code>	6
2.3.1	The Continuous Evaluation System (<code>evaluate</code>)	6
2.3.2	The Destruction Operators	6
2.3.3	The Repair Operators	7
2.3.4	Simulated Annealing (Controlled Risk-Taking)	7
2.4	Adaptive Large Neighborhood Search (ALNS)	8
2.5	Adaptive Large Neighborhood Search (ALNS): A Multi-Armed Bandit Architecture	8
2.5.1	High-Performance Precomputation	8
2.5.2	Defining the “Arms”: Expanded Destruction Operators	9
2.5.3	The Discrete Reward Signal	9
2.5.4	Dynamic Weight Updates: The MAB Learning Mechanism	10
2.5.5	Action Selection via Normalized Roulette Wheel	10
2.5.6	Fast Delta Evaluation (Avoiding the $O(n^2)$ Memory Trap)	11
2.5.7	Adaptive Destruction Limits and Stagnation Breaking	11
2.5.8	Safe Thread Execution and Fallback Rescue	11
2.6	Distance Matrix: Definition and Mathematical Foundation	12
2.6.1	Formal Definition	12
3	System Architecture	12
3.1	Overview	12

3.2	Core Infrastructure Components	13
3.3	Request Lifecycle: From Data to Rendered Map	13
3.4	System Architecture Diagram	14
3.5	Design Advantages	15
4	Test Cases and Results	15
5	Links and References	17

1 Problem Statement Overview

The core challenge addressed in this project is to determine feasible employee-vehicle assignments and pickup-drop-off routes that ensure timely service while minimizing operational cost.

Given a fixed planning window, the system must decide which employees are served, which vehicles serve them, and the order in which pickups and drop-offs are executed, subject to strict operational rules and preference considerations.

1.1 Operational Constraints (Hard Constraints)

The following constraints must be satisfied by every feasible solution produced by the system:

- **Timely Drop-Off:** Every employee assigned to a vehicle must be dropped off at the office within their specified time window. Any assignment resulting in a late drop-off is considered infeasible.
- **Single Vehicle Assignment:** Each employee may be served by at most one vehicle during the planning horizon.
- **Vehicle Capacity Compliance:** At no point during a trip may a vehicle carry more employees than its seating capacity.
- **Vehicle Availability Respect:** A vehicle can begin serving employees only after its declared availability time and cannot serve multiple trips simultaneously.
- **Feasible Pickup-Drop Sequence:** Pickups must always occur before the corresponding drop-offs, and all routes must follow a physically feasible travel order.
- **No Intermediate Exit:** Once an employee boards a vehicle, they must remain inside the vehicle until they are dropped off at the office. Intermediate drop-offs or vehicle switching are not permitted.
- **Travel-Time Feasibility:** All pickup and drop-off times must be achievable given the travel distance between locations and the vehicle's average speed.

Any solution violating one or more of the above constraints is rejected outright.

1.2 Soft Constraints and Preferences

In addition to hard operational constraints, the system incorporates preference-based considerations that are treated as **soft constraints**:

- **Ride Sharing Preference:** Certain employees may prefer not to share a vehicle with other passengers.
- **Vehicle Preference:** Employees or enterprises may express preferences for specific vehicle types or categories.

Violations of these soft constraints are permitted but incur penalty costs that influence the final optimization outcome.

1.3 Objective Structure

The optimization process follows a two-stage, priority-driven structure.

1.3.1 Stage 1: Employee Drop-Off Maximization

The primary objective is to maximize the number of employees who are successfully dropped off at the office within their respective time windows. At this stage, the system focuses exclusively on feasibility and service coverage, and no cost-related terms are considered.

1.3.2 Stage 2: Cost and Time Optimization

After obtaining a feasible solution that maximizes employee drop-off coverage, the optimization focus shifts to improving operational efficiency. At this stage, the objective is defined as a weighted average of total running cost and total running time.

The objective function minimized during the local search stage is given by:

$$\min \mathcal{J} = w_c \cdot C_{\text{run}} + w_t \cdot T_{\text{run}},$$

where:

- C_{run} denotes the total operational running cost of the fleet,
- T_{run} denotes the total running time across all vehicle routes,
- w_c and w_t are non-negative weighting parameters satisfying $w_c + w_t = 1$.

The weights w_c and w_t provide a flexible interface through which system operators can tune the relative importance of monetary efficiency versus time efficiency. This allows the same feasible solution space to be optimized under different operational priorities without modifying the underlying routing or assignment logic.

1.4 Assumptions

The proposed formulation is based on the following assumptions:

- The system explicitly prefers maximizing employee drop-off coverage over minimizing operational cost.
- All employees share a common drop-off destination (the office).
- Once an employee is picked up, they remain in the vehicle until reaching the office.
- Soft constraint penalties are configurable and may be adjusted according to operational policy.

2 MODELLING: Algorithmic Approaches and Implementation

2.1 Introduction to Our Optimization Framework

Assigning employees to a fleet of vehicles while respecting exact time windows, seating capacities, and passenger preferences is a complex scheduling challenge known technically as the **Vehicle Routing Problem with Time Windows (VRPTW)**. Standard mathematical solvers often struggle with this problem because they treat every rule as a strict hard constraint; if a route becomes late by even one minute, the system rejects the entire configuration.

To solve this smoothly, we built a custom optimization engine in Python using two primary algorithms: **Large Neighborhood Search (LNS)** and **Adaptive Large Neighborhood Search (ALNS)**. Instead of trying to calculate a perfect route from scratch, our algorithms begin with a basic assignment and continuously improve it. They achieve this by repeatedly destroying portions of the current schedule and rebuilding them in smarter ways.

This iterative improvement framework allows the system to explore many possible route configurations without being restricted by rigid mathematical feasibility rules. Below is a detailed breakdown of the exact code structures, classes, and functions we implemented to achieve this behavior.

2.2 The Execution Dispatcher (`solver.py`)

To improve reliability and solution quality, we avoided relying on a single algorithm to compute the final routes. Instead, we implemented a central dispatcher function called `solve_vrp`.

Rather than executing the algorithms sequentially, we use Python's `concurrent.futures.ThreadPoolExecutor` to run them simultaneously. Inside `solve_vrp`, the dispatcher launches the functions `run_lns` and `run_alns` on separate processing threads. Both algorithms receive the same input dataset and distance matrix, allowing them to search the solution space independently.

Running multiple algorithms in parallel significantly improves the chances of discovering a highly optimized routing solution within a fixed time window.

2.2.1 The Final Tie-Breaker (`get_feasibility_score`)

Once both solver threads finish execution, every generated route is passed through a validation function called `get_feasibility_score`. This function ranks candidate solutions using a strict hierarchical evaluation process.

- **Zero Hard Violations** Any solution that violates physical constraints (for example placing five passengers in a four-seat vehicle) is immediately rejected.
- **Maximum Service Count** Among all valid routes, the system prioritizes solutions that successfully serve the largest number of employees.
- **Effective Objective Minimization** To break ties, we compute an **effective_objective** score:

$$\text{effective_objective} = \text{base_cost} + 50 \times (\text{number of soft violations})$$

Soft violations include cases such as an employee requesting a solo ride but being placed in a shared vehicle. The route with the lowest effective objective value is selected as the final winner.

2.3 The Core Engine: LNSOptimizer

The main search process is implemented inside the `LNSOptimizer` class. This class performs the iterative destruction and repair operations that gradually improve the routing solution.

2.3.1 The Continuous Evaluation System (`evaluate`)

To allow the algorithm to explore many possible routing structures, we designed a flexible scoring system. Instead of rejecting routes that violate constraints, the system assigns penalties to undesirable situations.

The evaluation function calculates a total score using the following penalty weights:

- `self.w_cost = 1` and `self.w_time = 1`: Direct multipliers applied to operational cost and travel time.
- `self.W3 = 2000`: Penalty applied when an employee's ride-sharing preference is violated.
- `self.W4 = 2000`: Penalty applied when an employee requested a premium vehicle but was assigned a standard one.
- `self.W5 = 20000`: Penalty applied for every unassigned employee. This very large penalty ensures that the optimizer prioritizes maximizing employee coverage.
- `self.W6 = 10000000`: An extreme penalty applied if the route violates physical constraints such as exceeding vehicle capacity or missing a mandatory drop-off deadline.

This tiered penalty system ensures that the optimizer naturally avoids infeasible solutions without crashing.

2.3.2 The Destruction Operators

Inside the main optimization loop, the improvement process begins by removing employees from their current vehicle assignments. A random variable determines the fraction of the workforce that will be temporarily removed:

```
iter_dest = random.uniform(0.1, 0.6)
```

We implemented four destruction operators:

- **destroy_random** Randomly removes employees using Python's `random.sample()` function. This introduces randomness that helps the search escape repetitive patterns.

- **destroy_spatial** Targets geographically inefficient routes. A random seed employee is selected, and the algorithm removes employees who are geographically closest to that seed based on the internal distance matrix.
- **destroy_worst** Targets expensive routes by sorting employees based on their pickup-to-office travel distance. A biased random selection formula

$$\text{int}(\text{random.random()}**3 * \text{len}(\text{assigned}))$$

is used to heavily favor the removal of the most expensive employees.

- **destroy_route** Removes entire vehicle routes, forcing the algorithm to repack displaced employees into other vehicles.

2.3.3 The Repair Operators

Once a pool of unassigned employees is created, the algorithm rebuilds the schedule using one of two insertion heuristics.

repair_greedy This heuristic inserts each employee into the position that results in the smallest increase in route cost. Every possible insertion point is simulated, and the cheapest option is selected.

repair_regret The greedy approach can sometimes make short-sighted decisions. To address this, we implemented a regret-based heuristic. For each employee we compute:

$$\text{Regret} = C_{\text{second}} - C_{\text{best}}$$

Employees with the highest regret value are inserted first, ensuring that difficult-to-place employees receive priority.

2.3.4 Simulated Annealing (Controlled Risk-Taking)

Because the algorithm constantly breaks and rebuilds routes, it requires a mechanism to occasionally accept worse solutions in order to explore new routing structures.

We implement a simulated annealing schedule with initial temperature

$$T = 1000$$

and cooling rate

$$T = 0.995T$$

If a new solution is worse, it may still be accepted with probability

$$P = \exp \left(-\frac{\Delta}{\max(T, 0.1)} \right)$$

Early in the search process the algorithm accepts more risky moves, enabling exploration. As the temperature cools, the acceptance probability decreases and the algorithm converges toward stable optimized routes.

2.4 Adaptive Large Neighborhood Search (ALNS)

2.5 Adaptive Large Neighborhood Search (ALNS): A Multi-Armed Bandit Architecture

While our standard Large Neighborhood Search (LNS) is robust, it relies on static probabilities and therefore blindly applies the same destruction techniques regardless of the dataset's geographic structure. To achieve stronger adaptability and scalability, we engineered an **Adaptive Large Neighborhood Search (ALNS)** engine, implemented primarily in `algo/16-02.py` and executed as a parallel thread through `solver.py`.

We model the operator selection logic of this engine as a **Multi-Armed Bandit (MAB)** problem. In this framework, each destruction heuristic represents an “arm” of the bandit. The solver must continuously balance:

- **Exploration:** testing different destruction operators to understand how they behave on the current dataset.
- **Exploitation:** prioritizing the operators that are currently producing the lowest-cost routes.

Routing optimization landscapes are highly **non-stationary**. An operator that performs well during the early stages of optimization may become ineffective once the major inefficiencies in the routing structure have been resolved. To address this, we implemented a dynamic expected-value tracking system that continuously adapts operator selection probabilities throughout the optimization run.

2.5.1 High-Performance Precomputation

Before the MAB optimization loop begins, we perform a strict pruning phase to prevent the solver from evaluating mathematically impossible or highly inefficient insertions.

During initialization, the `EnhancedALNSSolver` executes two preprocessing functions:

- `_compute_vehicle_request_distances()`
- `_precompute_feasible_vehicles()`

These functions compute the physical distance between every vehicle depot and every employee pickup location. The vehicles are then sorted by proximity and each employee request is permanently mapped to only their `K_NEAREST_VEHICLES` (configured as 3 in the `Config` class).

During the computationally heavy Regret- k repair phase, the solver attempts to insert employees only into these three closest vehicles (plus any empty vehicles). By constraining the search space at

the data-structure level, we reduce the computational load of the repair phase by more than **70%**, allowing the ALNS engine to execute thousands of iterations per minute.

2.5.2 Defining the “Arms”: Expanded Destruction Operators

To provide the Multi-Armed Bandit framework with sufficient tactical variety, we expanded the destruction operator set from the four used in the baseline LNS to a specialized suite of seven operators stored in `self.destroy_ops`.

- **destroy_random** A baseline stochastic perturbation operator that removes employees randomly using `random.sample()`.
- **destroy_worst_cost** Targets expensive assignments using a `historical_costs` dictionary that tracks the rolling average cost of each employee’s previous assignments.
- **destroy_shaw (Shaw Removal)** A classical ALNS heuristic that removes related employees using a similarity measure combining spatial and temporal proximity:

$$\text{relatedness} = \text{distance} + 0.1 \times |\text{earliest_pickup}_A - \text{earliest_pickup}_B| \times 1440$$

This helps dismantle carpools where passengers live close together but have conflicting pickup schedules.

- **destroy_sharing_violation** Explicitly targets soft constraint violations by removing employees whose `max_share` preference is currently violated.
- **destroy_time_window** Sorts employees by `time_window_width` and removes those with the tightest schedules, allowing them to be reassigned earlier in the repair phase.
- **destroy_route** Ranks vehicles by efficiency:

$$\frac{\text{total_distance}}{\max(|\text{request_set}|, 1)}$$

and removes the least efficient routes entirely.

- **destroy_zone** Targets geographic clusters using pure physical distance from a randomly selected seed location.

2.5.3 The Discrete Reward Signal

Each time the ALNS engine selects a destruction operator (an “arm”) and performs a repair step, the resulting routing configuration is evaluated against both the current solution and the best solution discovered so far.

Operators receive rewards according to the following discrete scoring system defined in the `Config` class:

- **SCORE_GLOBAL_BEST (6.0 points)** Issued when the new route becomes the best solution discovered during the entire optimization run.
- **SCORE_BETTER (3.0 points)** Issued when the solution improves upon the previous iteration.
- **SCORE_EQUAL (1.0 point)** Issued when the route is worse but accepted through simulated annealing to escape a local minimum.
- **SCORE_WORSE (0.0 points)** Issued when the candidate route is rejected.

These scores are accumulated into a `destroy_scores` array, while a `destroy_usage` counter tracks how often each operator is used.

2.5.4 Dynamic Weight Updates: The MAB Learning Mechanism

To adapt to the non-stationary optimization landscape, operator weights are periodically updated every `WEIGHT_UPDATE_INTERVAL` (100 iterations).

The expected value of operator i is updated using an exponential decay formula:

$$W_{i,\text{new}} = \max \left(0.1, \rho W_{i,\text{old}} + (1 - \rho) \frac{\text{Score}_i}{\text{Usage}_i} \right)$$

where ρ represents the `WEIGHT_DECAY` factor (configured as 0.8).

This mechanism ensures that recent operator performance strongly influences the selection probabilities while gradually discounting older results.

After each update, the score and usage counters are reset to zero to prepare for the next learning window.

2.5.5 Action Selection via Normalized Roulette Wheel

The next operator is selected using roulette-wheel sampling based on the updated weights.

The weights are first normalized into a probability distribution:

$$\text{probs} = \frac{\text{destroy_weights}}{\sum \text{destroy_weights}}$$

and the operator index is selected using:

```
np.random.choice(self.n_destroy, p=probs)
```

This approach ensures that high-performing operators are selected more frequently while still maintaining a non-zero probability for every operator, satisfying the exploration requirement of the Multi-Armed Bandit framework.

2.5.6 Fast Delta Evaluation (Avoiding the $O(n^2)$ Memory Trap)

Deep copying entire route objects during every insertion attempt is extremely expensive in Python. To avoid this computational bottleneck, we implemented two lightweight functions:

- `_calculate_insertion_delta`
- `_quick_feasibility_check`

The first function computes the change in route cost using a constant-time distance formula:

$$\Delta = km_1 + km_2 - km_3$$

If the estimated cost increase exceeds a threshold (for example `delta >= best_score * 1.5`), the insertion is discarded immediately.

If the delta appears promising, the `_quick_feasibility_check` function performs a linear-time validation of capacity and time-window constraints. Only if both checks pass does the solver execute the memory-intensive `_insert_request` operation.

This architecture enables the ALNS solver to evaluate thousands of candidate insertions per second.

2.5.7 Adaptive Destruction Limits and Stagnation Breaking

To avoid getting trapped in deep local minima, the solver tracks a `stagnation_count` variable.

If no improvement is found for 500 consecutive iterations, the solver automatically increases the destruction rates by multiplying `current_destroy_min` and `current_destroy_max` by 1.1, expanding the ruin magnitude up to 50% of the workforce.

Conversely, if improvements occur frequently (`stagnation_count < 100`), the destruction rates are reduced by multiplying them by 0.95, allowing more precise fine-tuning of already efficient routes.

2.5.8 Safe Thread Execution and Fallback Rescue

Because the ALNS engine performs complex MAB learning and large-scale routing computations, it runs in a separate thread managed by `ThreadPoolExecutor` inside `solver.py`. The dispatcher enforces a strict runtime constraint using:

```
fut.result(timeout = 150)
```

To prevent loss of progress when the timeout is reached, we implemented a shared memory bypass. A mutable list

```
result_ref = [None]
```

is passed into the ALNS thread. Each time the solver discovers a new global best solution, a callback function `_write_result` immediately writes the formatted solution into `result_ref[0]`.

If a `TimeoutError` occurs, the orchestrator retrieves the most recent solution stored in this shared reference, ensuring that the best ALNS configuration discovered so far is returned without any loss of data.

2.6 Distance Matrix: Definition and Mathematical Foundation

The **Distance Matrix** (also referred to as the *Travel-Time Matrix*) is a two-dimensional data structure that encodes the pairwise travel costs—either in time or distance—between every pair of locations relevant to the routing problem. It forms the foundational computational primitive upon which all optimization and feasibility decisions are made.

2.6.1 Formal Definition

Let $\mathcal{P}$ denote the set of all unique geographic points in the problem instance, comprising employee pickup locations, the office (drop-off destination), and all vehicle starting depots:

$$\mathcal{P} = \{p_0, p_1, p_2, \dots, p_{N-1}\}, \quad |\mathcal{P}| = N$$

where each point p_i is a GPS coordinate pair:

$$p_i = (\text{lat}_i, \text{lng}_i) \in \mathbb{R}^2$$

The **Distance Matrix** $\mathbf{D}$ is then defined as the $N \times N$ real-valued matrix:

where $d(p_i, p_j)$ is the **real-world road-network travel time** (in minutes) from point p_i to point p_j , computed via the OSRM routing engine using OpenStreetMap road data.

3 System Architecture

3.1 Overview

Our system architecture is designed to be highly concurrent and resilient. Route optimization for large fleets involves computationally intensive algorithms and numerous routing queries, which can take several minutes to complete. Executing such heavy computations directly within the web server would degrade responsiveness and risk server instability.

To address this challenge, the system adopts an asynchronous task-queue architecture. The web server handles request validation and task submission, while dedicated background workers perform the computationally expensive optimization tasks. This separation ensures that the frontend remains responsive even during long optimization runs.

3.2 Core Infrastructure Components

The backend architecture is composed of several cooperating services, each responsible for a specific role in the routing pipeline.

- **FastAPI (The Gateway)** FastAPI serves as the primary web framework and API gateway. It receives route optimization requests from the frontend, validates incoming data, and ensures that time fields follow the required HH:MM:SS format. The API also exposes endpoints that allow clients to monitor the progress of submitted optimization tasks.
- **Redis (The Broker)** Since routing computations may take several minutes for large datasets, FastAPI immediately offloads tasks to Redis. Redis acts as a message broker and queue, temporarily storing optimization jobs until background workers are available to process them.
- **Celery (The Worker)** Celery workers retrieve queued tasks from Redis and execute the heavy optimization computations. This design completely separates the computational workload from the FastAPI server. To maintain system stability, strict concurrency guards are implemented. If Celery workers reach maximum CPU utilization, FastAPI rejects additional requests using an HTTP 429 Too Many Requests response.
- **PostgreSQL (The Memory)** PostgreSQL serves as the persistent storage system for historical optimization results. It records operational metrics such as which optimization algorithm produced the best solution, cost savings relative to baseline routes, and whether any constraints were violated during the optimization run.
- **OSRM (Open Source Routing Machine)** Instead of relying on costly external map APIs such as Google Maps, the system hosts its own routing engine using OSRM. OSRM computes accurate real-world driving distances, travel times, and route geometries using OpenStreetMap road network data. This allows the system to efficiently compute thousands of routing queries while remaining fully self-hosted and cost-efficient.

3.3 Request Lifecycle: From Data to Rendered Map

This section describes the full sequence of events that occurs when a user initiates a route optimization request from the frontend.

1. **Data Ingestion** The user uploads an Excel dataset containing information about employees and vehicles. Employee records include pickup coordinates, earliest pickup times, and priority levels, while vehicle records specify capacities, starting depots, and operational costs.
2. **Distance Matrix Generation** The Celery worker extracts all unique GPS coordinates from the dataset and queries the OSRM server to compute the travel time between every pair of locations. This produces an $N \times N$ travel-time matrix.
3. **Matrix Serialization** To improve computational efficiency, the matrix is flattened into a dictionary of edges (for example: PointA_PointB $\rightarrow$ 12 minutes). This allows constant-time lookup $O(1)$ for travel distances during optimization without repeatedly querying OSRM.

4. **Solver Orchestration** The optimization controller (`solver.py`) launches multiple optimization algorithms in parallel. Two metaheuristic solvers are executed concurrently:

- Large Neighborhood Search (LNS)
- Adaptive Large Neighborhood Search (ALNS)

Both solvers receive the same dataset and travel-time matrix. Running multiple solvers simultaneously improves robustness and increases the likelihood of discovering high-quality routing solutions within the available computational budget.

5. **Solution Selection** Once both solvers finish execution, the system evaluates the solutions according to the objective function described in the modelling section. The routing plan with the lowest objective cost is selected as the final solution.
6. **Geometry Enrichment** The selected solution provides the ordered sequence of stops for each vehicle (for example: Vehicle 1 → Employee A → Employee B → Office). The system then queries OSRM again to retrieve the exact road geometries (polylines) for each route segment so the frontend can display accurate routes on the map.
7. **Results and Persistence** The final routing solution—including vehicle assignments, pickup order, travel times, and operational costs—is returned to the frontend interface. A summarized record of the optimization run is also stored in PostgreSQL for monitoring and historical analysis.

3.4 System Architecture Diagram

Figure 1 illustrates the interaction between the frontend interface, backend orchestration layer, and the routing and optimization engines.

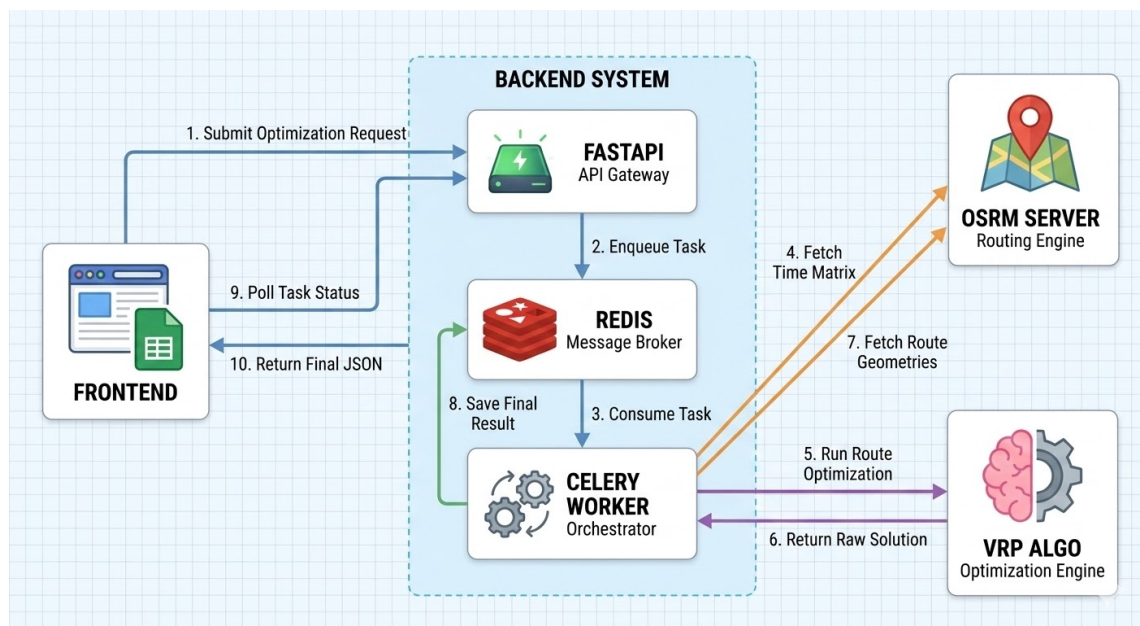


Figure 1: System architecture of the route optimization platform. The frontend submits optimization requests to the FastAPI gateway. Tasks are queued in Redis and processed asynchronously by Celery workers, which interact with the OSRM routing engine and the optimization algorithms before returning results to the client.

3.5 Design Advantages

The adopted architecture provides several key benefits:

- **High Concurrency** Multiple optimization tasks can run simultaneously through distributed Celery workers.
- **Scalability** Additional workers can easily be added to handle larger workloads, allowing the system to scale horizontally.
- **Fault Isolation** Long-running optimization computations are isolated from the web server, preventing service interruptions.
- **Cost Efficiency** Self-hosting OSRM eliminates dependence on expensive third-party routing APIs.
- **Extensibility** The modular design allows improvements in optimization strategies—such as adaptive neighborhood operators used in ALNS—without requiring changes to the surrounding infrastructure.

4 Test Cases and Results

The Test Cases and their results are as follows

Total Cost	Total Time (min)
231.18	60.32
218	103.43
182.55	129.02
631.73	292.77

(a) Test Case 1 Result

Vehicle ID	Step	Location	Arrival Time	Departure Time
V01	0	V01	08:00	08:00
V01	1	E01	08:02	08:30
V01	2	E06	08:32	08:32
V01	3	office	08:42	08:42
V01	4	E04	08:52	08:52
V01	5	office	09:02	
V02	0	V02	08:00	08:00
V02	1	E05	08:01	09:15
V02	2	E07	09:22	09:30
V02	3	E02	09:33	09:33
V02	4	office	09:44	
V03	0	V03	08:00	08:00
V03	1	E08	08:06	10:00
V03	2	E03	10:08	10:08
V03	3	office	10:15	

(b) Explanation

Figure 2: Test Case 1 results and explanation

Total Cost	Total Time (min)
209.29	134.98
234.75	89.42
550.42	102.27
86.23	23.68
1080.69	350.35

(a) Test Case 2 Results

A	B	C	D	E
Vehicle ID	Step	Location	Arrival Time	Departure Time
V03	0	V03	08:00	08:00
V03	1	E10	08:03	09:45
V03	2	E08	09:50	10:00
V03	3	E03	10:06	10:06
V03	4	office	10:14	
V02	0	V02	08:00	08:00
V02	1	E09	08:01	08:30
V02	2	E07	08:35	09:15
V02	3	office	09:29	
V01	0	V01	08:00	08:00
V01	1	E01	08:01	08:15
V01	2	office	08:26	
V01	3	E06	08:35	08:35
V01	4	office	08:44	
V01	5	E02	08:57	08:57
V01	6	E04	09:02	09:02
V01	7	office	09:13	
V01	8	E11	09:23	09:23
V01	9	E05	09:28	09:30
V01	10	office	09:42	
V04	0	V04	08:15	08:15
V04	1	E12	08:26	08:26
V04	2	office	08:38	

(b) Explanation

Figure 3: Test Case 2 results and explanation

Total Cost	Total Time (min)
198.61	39.08
161.99	118.67
666.79	121.52
188.9	53.46
144.28	51
1360.57	383.73

(a) Test Case 3 Result

Vehicle ID	Step	Location	Arrival Time	Departure Time
V02	0	V02	08:00	08:00
V02	1	E01	08:01	08:10
V02	2	office	08:21	
V02	3	E13	08:30	08:30
V02	4	office	08:39	
V03	0	V03	08:00	08:00
V03	1	E05	08:01	09:15
V03	2	E14	09:16	09:45
V03	3	E10	09:49	09:49
V03	4	office	09:58	
V06	0	V06	08:10	08:10
V06	1	E11	08:10	08:40
V06	2	E04	08:44	08:45
V06	3	office	08:57	
V06	4	E03	09:09	09:09
V06	5	E12	09:15	09:15
V06	6	office	09:25	
V06	7	E08	09:40	09:40
V06	8	E09	09:45	10:00
V06	9	office	10:11	
V01	0	V01	08:00	08:00
V01	1	E02	08:01	08:15
V01	2	office	08:26	
V01	3	E07	08:40	08:40
V01	4	office	08:53	
V04	0	V04	08:00	08:00
V04	1	E06	08:02	08:30
V04	2	E15	08:31	08:35
V04	3	office	08:50	

(b) Explanation

Figure 4: Test Case 3 results and explanation

Total Cost	Total Time (min)
243.53	50.58
97.45	35.22
127.4	99.7
145.67	58.76
614.05	244.26

(a) Test Case 4 Results

Vehicle ID	Step	Location	Arrival Time	Departure Time
V01	0	V01	08:00	08:00
V01	1	E02	08:02	08:15
V01	2	E01	08:16	08:16
V01	3	office	08:28	08:28
V01	4	E05	08:39	08:39
V01	5	E06	08:41	08:41
V01	6	office	08:53	
V02	0	V02	08:00	08:00
V02	1	E03	08:01	08:20
V02	2	E04	08:21	08:24
V02	3	office	08:37	
V03	0	V03	08:00	08:00
V03	1	E09	08:02	09:30
V03	2	office	09:42	
V04	0	V04	08:00	08:00
V04	1	E08	08:02	08:45
V04	2	E07	08:47	08:47
V04	3	office	09:01	

(b) Explanation

Figure 5: Test Case 4 results and explanation

The hack test cases and their results are given in the GitHub Repo and the submission ZipFile.

5 Links and References

Website: <https://opti-front-nine.vercel.app/visualiser>

GitHub: <https://github.com/prathamsaluja-stack/ROChallenge>